

MoneyTransferApp

Application mobile de transfert d'argent

Flutter

Laravel 12

Ollama

Stripe

Algorand

Louisy David · L3 Informatique

1. Présentation du projet

Qu'est-ce que MoneyTransferApp ?

Fonctionnalités	Description
Transfert	Envoyer de l'argent à un autre utilisateur
Recharge	Recharger son compte par carte bancaire
QR Code	Payer en scannant un QR Code (TTL 10s)
Historique	Consulter ses transactions avec preuve blockchain
Chatbot	Assistant IA (Ollama LLaMA 3.2)
Profil	Gérer ses informations et son PIN
Mot de passe oublié	Code 8 chiffres, expiration 15 minutes
Notifications	Alertes email automatiques par événement

2. Backend - Laravel 12

Stack technique

Framework	Laravel 12 (PHP 8.2)
Base données	PostgreSQL 16
Cache	Redis 7
Auth	Laravel Sanctum
Endpoints	18 routes REST
Soldes	Stockés en centimes
Transferts	Transaction SQL atomique

Endpoints API (/api/v1/)

POST	auth/register	- Inscription
POST	auth/login	- Connexion → Token
POST	auth/logout	- Déconnexion
GET	auth/me	- Profil courant
POST	auth/forgot-password	- Reset (code 8 chiffres)
POST	transfer	- Transfert (PIN requis)
POST	recharge/create-intent	- Stripe PaymentIntent
POST	recharge/webhook	- Stripe Webhook signé
GET	history	- Historique paginé
POST	qr/generate	- Générer QR Code
POST	qr/scan	- Scanner et payer
POST	chatbot/message	- Ollama IA

3. Infrastructure - Docker

Container	Image	Rôle	Port
app	PHP 8.2-FPM	Laravel API	:9000
nginx	nginx:1.25-alpine	Serveur web	:8000
postgres	postgres:16-alpine	Base de données	:5432
redis	redis:7-alpine	Cache / Sessions	:6379
ollama	ollama:latest	Chatbot IA LLaMA 3.2	:11434
pgadmin	pgadmin4:latest	Interface DB	:5400

Démarrage en une commande : `./setup.sh`

4. Architecture globale

APPLICATION FLUTTER (Android)

Login · Dashboard · Envoyer · QR Code · Chatbot · Historique · Profil · Mot de passe oublié

HTTPS + Bearer Token

API LARAVEL 12 (Backend)

Auth · Transfert · Recharge · Historique · QR Code · Chatbot · Reset Password

PostgreSQL · Redis · Ollama LLaMA 3.2 · Mailtrap SMTP

STRIPE

Paiements par carte bancaire
Webhook → Crédit automatique

ALGORAND

Blockchain testnet
Chaque transfert enregistré on-chain

5. Sécurité - 6 couches de protection

Couche 1	Bearer Token Sanctum Authentification - accès interdit sans token valide
Couche 2	PIN bcrypt (6 chiffres) Autorisation → transfert impossible sans PIN, même si le token est volé
Couche 3	Webhook Stripe signé Anti-fraude → vérification HMAC-SHA256 de chaque webhook
Couche 4	QR Code TTL 10s + usage unique Anti-replay → impossible de réutiliser un QR Code expiré ou déjà scanné
Couche 5	Blockchain Algorand Immuabilité → aucune transaction ne peut être falsifiée ou effacée
Couche 6	Reset password sécurisé Code 8 chiffres bcrypt · expiration 15 min · usage unique · anti-énumération

6. Frontend Flutter - 10 écrans

Architecture

Pattern : Provider (State Management)

ApiService : appels HTTP + Bearer Token

AuthProvider : état de connexion

TransactionProvider : historique

Au démarrage → vérification du token

Token valide → Dashboard direct

Token invalide → LoginScreen

10 écrans développés

Login Connexion automatique si token existant

Register Inscription avec PIN obligatoire

Mot de passe oublié Email → code par mail

Reset password Code + nouveau mot de passe

Dashboard Solde + actions + pull-to-refresh

Envoyer Email + montant + PIN obligatoire

Recharge Payment Sheet Stripe native

Historique Badge Algorand cliquable → Lora

QR Code Génération TTL 10s + scanner

Chatbot / Profil LLaMA 3.2 - Modifier infos & PIN

7. Stripe - Le paiement par carte

- 1 Flutter demande la création d'un PaymentIntent au backend Laravel
- 2 Laravel crée le PaymentIntent via l'API Stripe → reçoit le client_secret
- 3 Flutter affiche la Payment Sheet native Stripe (saisie carte sécurisée)
- 4 Les données de carte vont directement chez Stripe (jamais sur le serveur)
- 5 Stripe confirme le paiement → envoie un Webhook signé à Laravel
- 6 Laravel vérifie la signature HMAC et crédite automatiquement le compte
- 7 Email de confirmation envoyé automatiquement avec le nouveau solde

Carte de test : 4242 4242 4242 4242 • Date : future • CVC : 123

8. Blockchain - Algorand

Pourquoi Algorand ?

Créateur	Silvio Micali (MIT, Prix Turing)
Consensus	Pure Proof of Stake
Vitesse	~4 secondes par bloc
Finalité	Immédiate (pas de rollback)
Frais	~0,001 € par transaction
Usage	CBDC, USDC, finance institutionnelle

Comment ça fonctionne ?

1. Transfert effectué en base SQL
2. Construction TX Algorand (note = {alice→bob, 1€, ref, ts})
3. Signature Ed25519 (sodium PHP)
4. Soumission à AlgoNode testnet
5. TX ID stocké + affiché dans Flutter
6. Badge 'Vérifié sur Algorand' cliquable

Transaction vérifiable sur Lora

TX ID	COBUHLC5RRW4EO3FHS3K...
Block	61237003
Timestamp	08 March 2026 18:09:54
Fee	0.001 ALGO
Note	{"app":"MoneyTransferApp","type":"transfer",...}

9. Notifications Email - Mailtrap SMTP

4 emails automatiques déclenchés par les événements :

Transfert reçu

→ Destinataire

Montant reçu, expéditeur, référence transaction

Transfert envoyé

→ Expéditeur

Virement effectué, nouveau solde, destinataire

Recharge Stripe confirmée

→ Utilisateur

Compte crédité, nouveau solde, référence Stripe

Mot de passe oublié

→ Utilisateur

Code 8 chiffres bcrypt, expiration 15 minutes

Flux : Alice envoie 10€ à Bob → Email 1 (Bob) → sleep(1) → Email 2 (Alice)

10. Mot de passe oublié - Flux sécurisé

- 1 Utilisateur saisit son email dans l'app Flutter
- 2 Backend génère un code à 8 chiffres aléatoire
- 3 Code hashé en bcrypt et stocké (TTL 15 minutes)
- 4 Email HTML stylisé envoyé via Mailtrap SMTP
- 5 Utilisateur saisit le code + nouveau mot de passe
- 6 Backend vérifie : non expiré + non utilisé
- 7 Mot de passe mis à jour + tous les tokens révoqués

✓ Usage unique · ✓ Expire 15 min · ✓ Anti-énumération · ✓ Révocation tous tokens

11. Difficultés rencontrées

Stripe - Configuration Android

Le SDK flutter_stripe exige que MainActivity hérite de FlutterFragmentActivity et que le thème utilise Theme.MaterialComponents. Sans ces deux configurations, l'application crashait au démarrage avec une PlatformException.

Algorand - Absence de SDK PHP compatible

Le SDK PHP officiel est incompatible avec Laravel 12. Implémentation manuelle de la signature Ed25519 avec l'extension sodium et l'encodage MessagePack. Les champs binaires doivent être encodés en type Bin et non en String.

Docker - Variables d'environnement non rechargées

Les nouvelles clés dans le .env n'étaient pas prises en compte après un simple docker-compose restart. Solution : --force-recreate pour que le container recharge les variables depuis le fichier .env.

Mailtrap - Limite de débit sandbox

Mailtrap sandbox autorise 1 email par seconde maximum. L'envoi simultané de 2 emails retournait une erreur 550. Solution : sleep(1) entre les deux envois + try/catch pour ne jamais bloquer la réponse API.

Port Docker - Ports réservés par Windows

Le port 5050 (PgAdmin) était dans une plage réservée par Windows (5041-5240), causant une erreur bind. Solution : migration vers le port 5400, libre de tout conflit.